GENERATIVE AI INTERNSHIP – Day 1

Topic: FastAPI

Submitted by: Shakthi K

Date: 01-10-2026

Introduction to APIs and Backend Development

I started by understanding the basic role of APIs in backend development. I learned how the frontend communicates with the backend by sending requests and receiving responses through APIs.

Basic flow:

Frontend → API Request → Backend → Processing → API Response → Frontend

HTTP Methods

I studied the commonly used HTTP methods and their basic purposes:

- **GET** – Retrieve data
- **POST** – Send or create data
- **PUT** – Update data
- **DELETE** – Remove data

I also understood that data sent through APIs is commonly handled in JSON format.

Request and Response

I learned how a backend receives data through a request, processes it, and sends the result back as a response.

Example request:

```
{
   "name": "Shakthi",
   "course": "FastAPI"
}
```

Example response:

{

"message": "Data received successfully",

"name": "Shakthi"

}

JSON

I studied JSON as a common format for exchanging data between frontend and backend. I understood its basic key-value structure and how it is used in API requests and responses.

Introduction to FastAPI

I learned that **FastAPI is a Python framework used to build APIs and backend applications**. I studied how to create a FastAPI application, define routes, handle requests, and return responses.

A basic FastAPI application uses:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")

def home():

    return {"message": "FastAPI is working"}
```

Running a FastAPI Application

I learned that **Uvicorn is used to run the FastAPI application** during development.

uvicorn main:app --reload

I also learned about the local FastAPI server and its automatically generated Swagger documentation.

FastAPI Architecture

I studied the basic architecture of a FastAPI backend and understood how different components work together.

Basic flow:

Client → FastAPI → Router → Validation → Service/Business Logic → Database or External API → Response

I understood that a larger application can separate these responsibilities into different modules and folders instead of keeping everything in one file.

Main Layers Studied

Router – Handles API endpoints and requests.

Schema – Defines and validates request and response data.

Service – Contains the main business logic.

Database – Handles database-related operations.

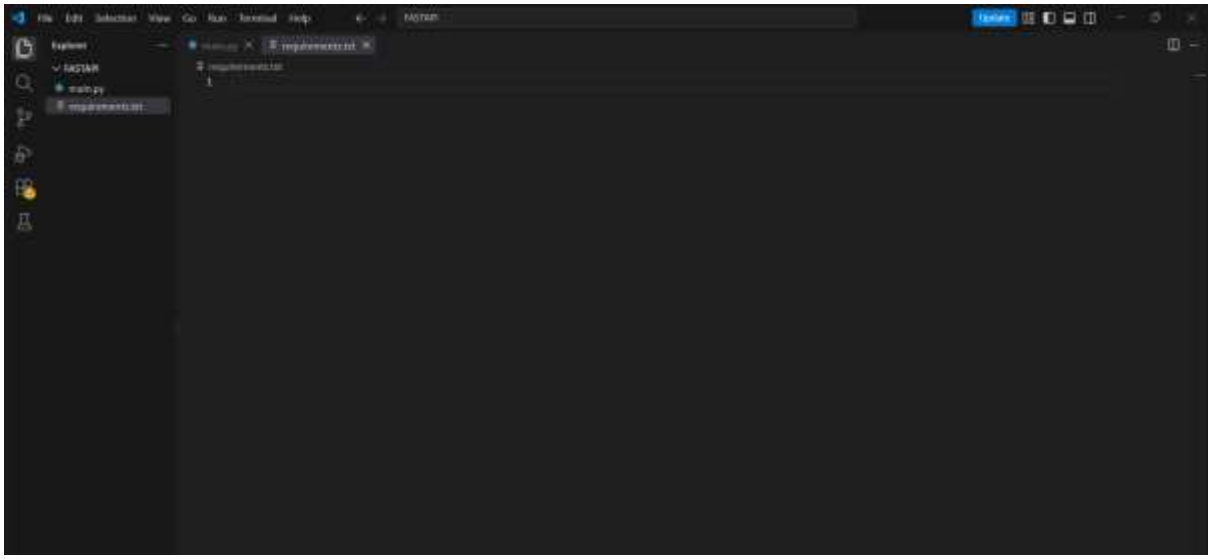
Models – Represents the application's data structure.

Utils – Contains reusable helper functions.

Through this study, I got a basic understanding of how FastAPI is structured and how it can be used to build a properly organized backend application.

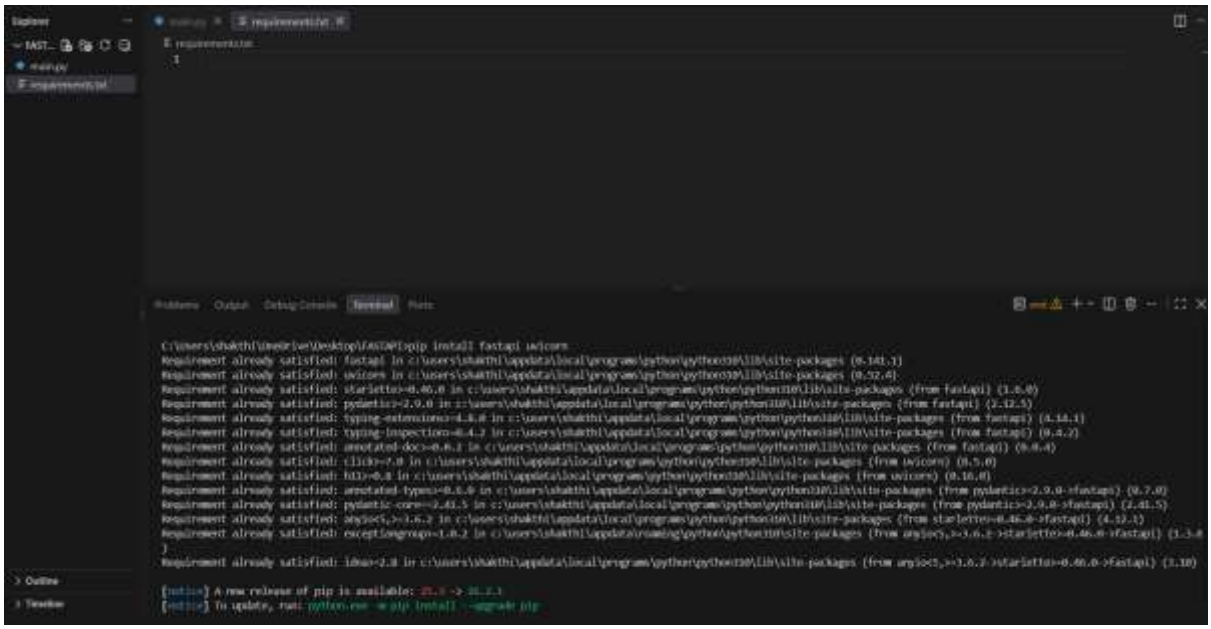
FastAPI Project Structure

The basic FastAPI project was created in VS Code with the required Python files and dependencies.



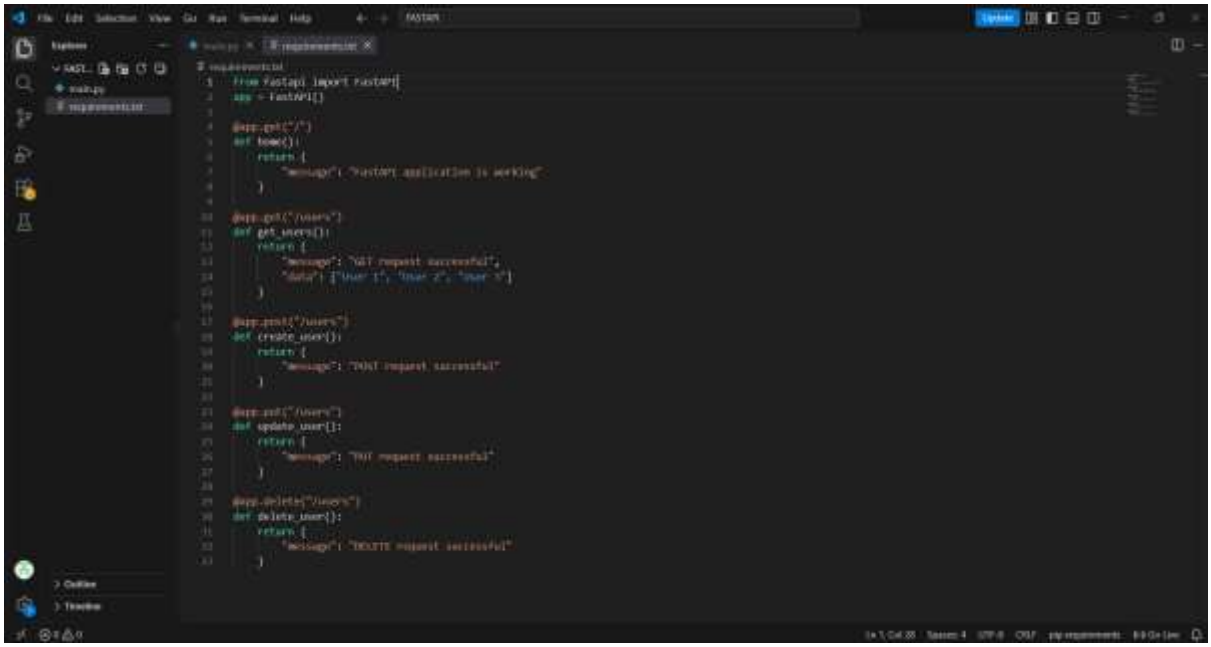
Installing FastAPI Dependencies

FastAPI and Uvicorn were installed to develop and run the backend API application.



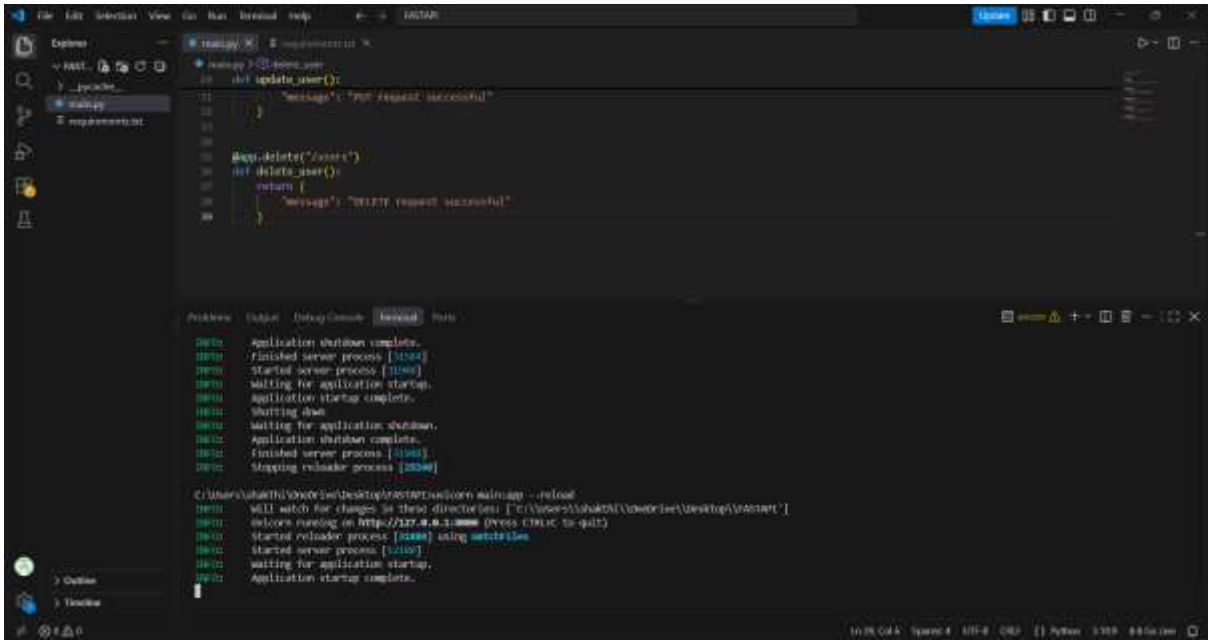
FastAPI HTTP Methods Implementation

Four commonly used HTTP methods — GET, POST, PUT and DELETE — were implemented in a single FastAPI application to understand their basic usage in backend API development.



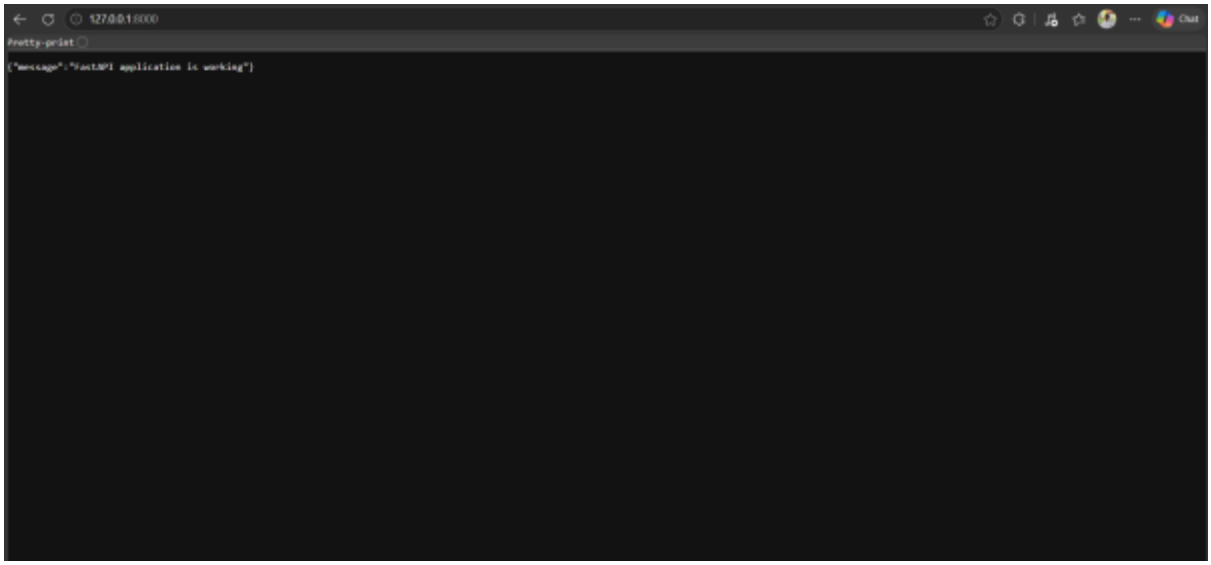
Running the FastAPI Server

The FastAPI application was successfully started using Uvicorn on the local development server.



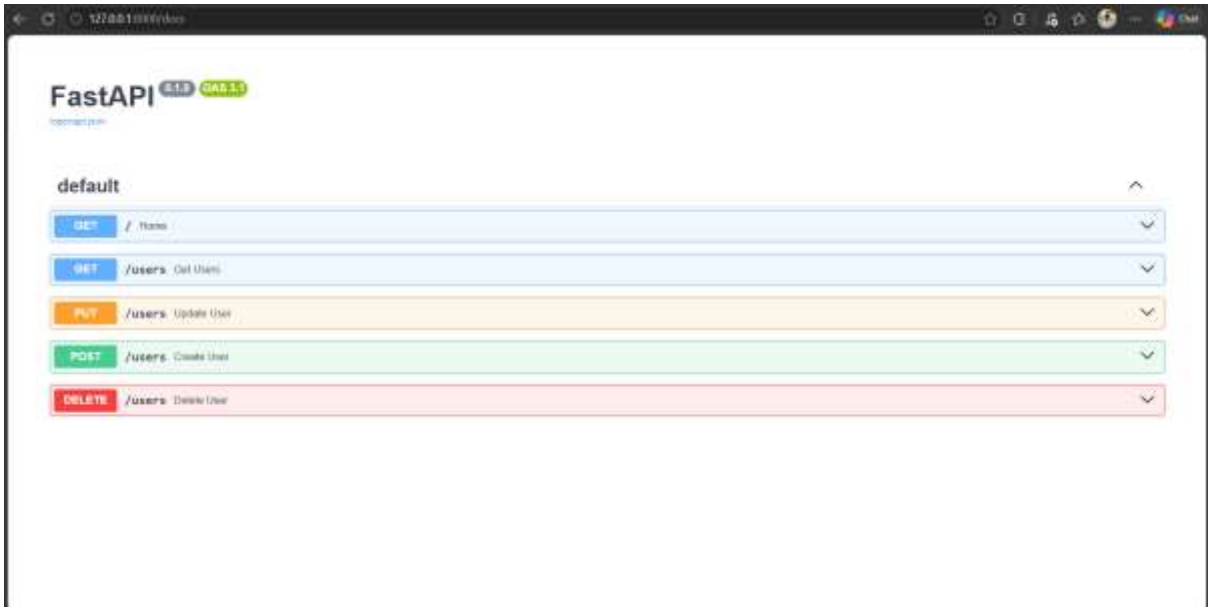
Testing the Root API Endpoint

The root endpoint was accessed through the browser, and the FastAPI backend successfully returned a JSON response.



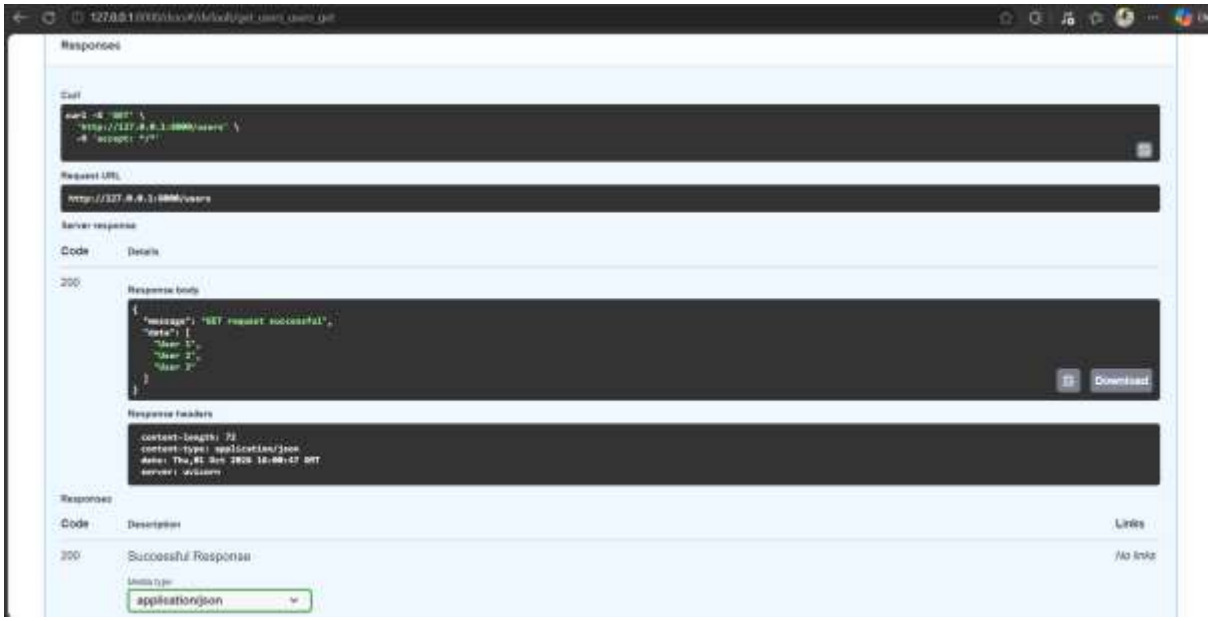
FastAPI Swagger UI

FastAPI automatically generated an interactive API documentation page where the implemented endpoints can be viewed and tested.



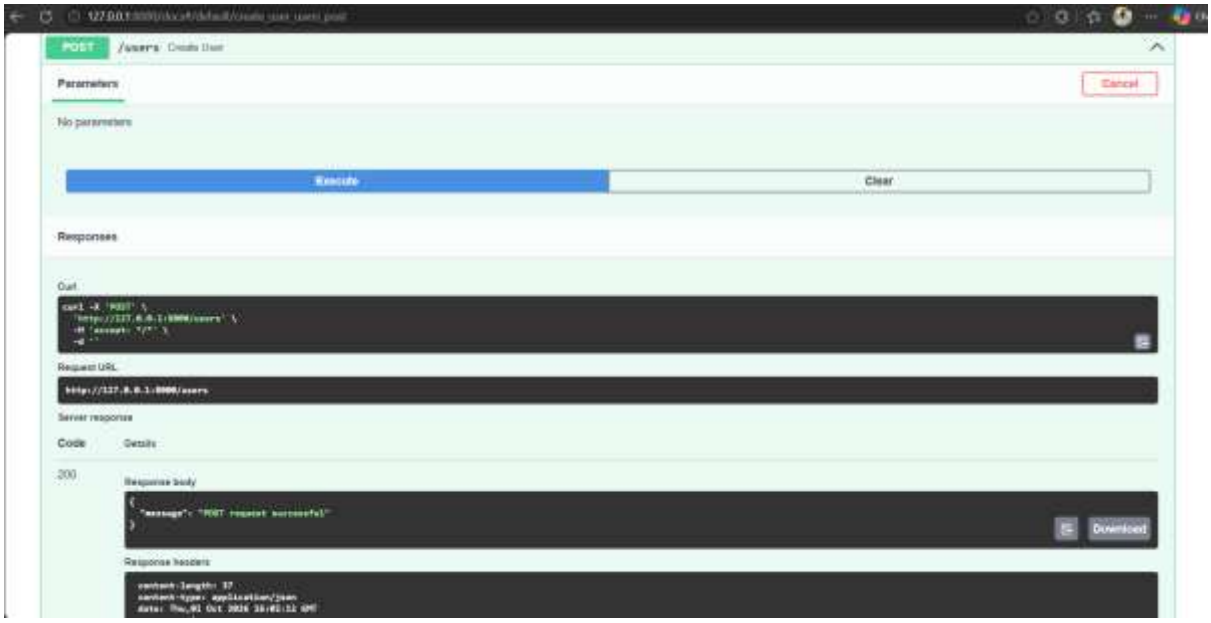
GET API Testing

The GET endpoint was tested through Swagger UI and successfully returned the requested user data from the backend.



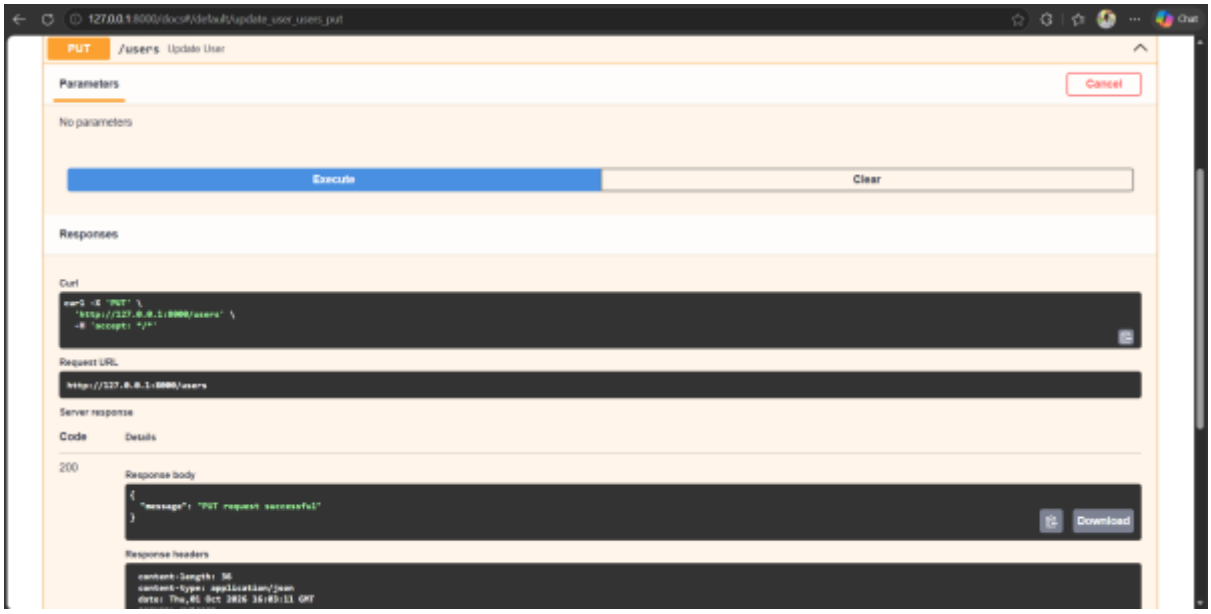
POST API Testing

The POST endpoint was tested through Swagger UI and successfully returned a response from the FastAPI backend.



PUT API Testing

The PUT endpoint was tested through Swagger UI to understand the basic structure of an update request and verify the response returned by the backend.



DELETE API Testing

The DELETE endpoint was tested through Swagger UI to understand the basic structure of a delete request and verify the response returned by the backend.

